1.1 Application Objectives

Smart File & Media Manager is a desktop application created using Python and Qt/PyQt6. The primary goal is to provide an easy-to-use Graphical User Interface (GUI) that enables users to organize, view, and analyze their local files.

When managing large collections of files, it can take a lot of time to review each individual file's characteristics. Typically, this would require manual review of all file types and file sizes. To assist users with these activities, this application provides all file information at once and offers additional features such as search capabilities, filter options, image/media preview capability and detailed file analysis.

This application was designed for students, office workers and everyday computer users whose job duties involve the management of files such as documents, photos/images, audio files etc.

In order to enhance the performance of the application, the program will include background processing when scanning large folders so that the user may continue to interact with the GUI while waiting for a scan to complete.

1.2 Functionality Requirements

Below are the 8 functionality requirements that describe what we expect the application to do. ID describes what function it performs.

FR01 The user selects the desired directory on their computer and loads it into the application.

FR02 The application displays the imported files including; file name, file type, size, and location.

FR03 The user searches for files based upon either the file name or file type.

FR04 The user previews supported images and plays supported audio files.

FR05 The application processes/multi-file scans utilizing background processing.

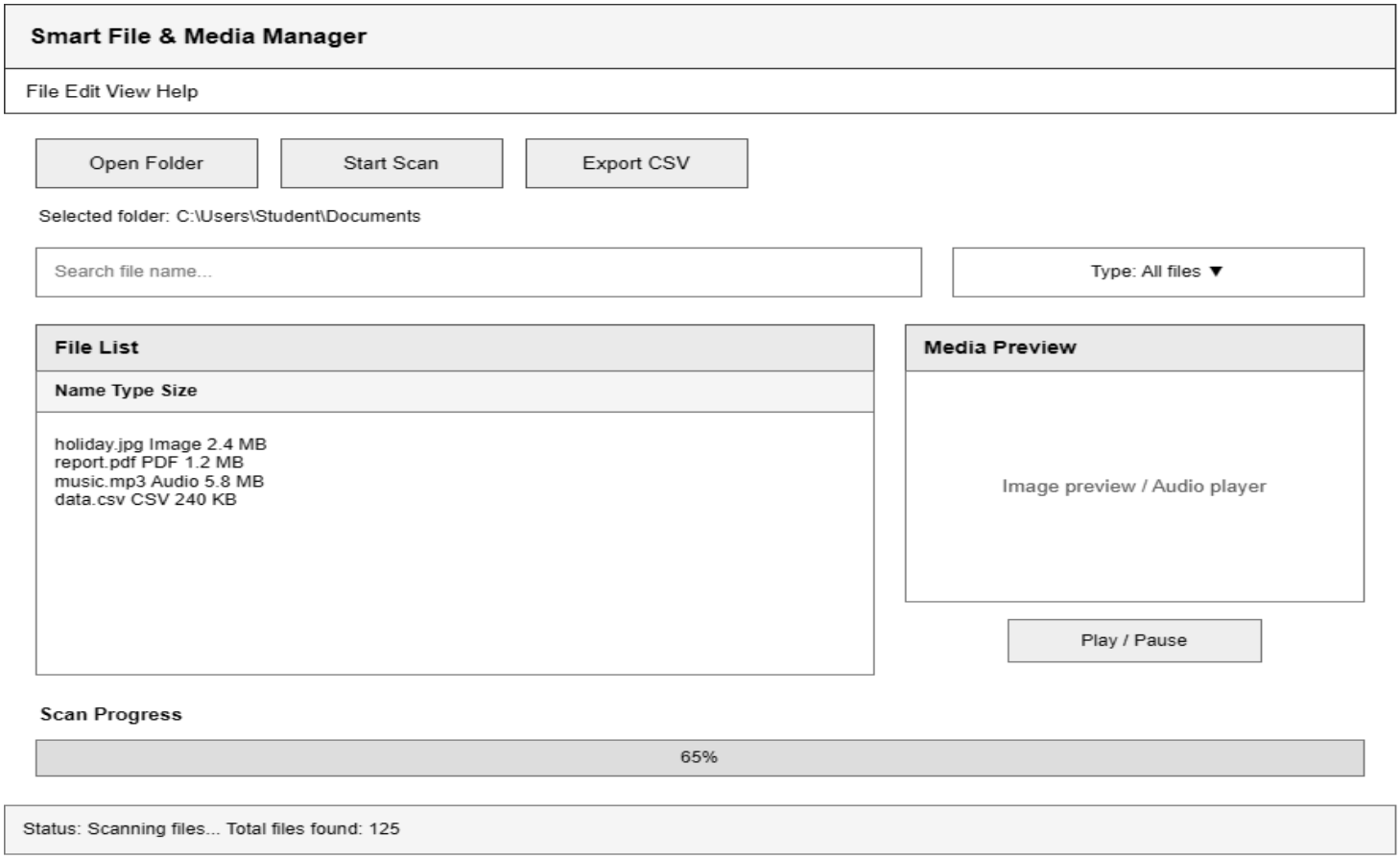
FR06 The application displays status updates during multi-file scanning operations, then notifies the user once completed.

FR07 The application responds to invalid files, inaccessible directories and other errors with a descriptive message.

FR08 The user exports a report detailing analyzed files into a comma separated value (.csv) file.

These requirements will serve as guides for the coding and testing phases in subsequent tasks.

1.3 GUI Design



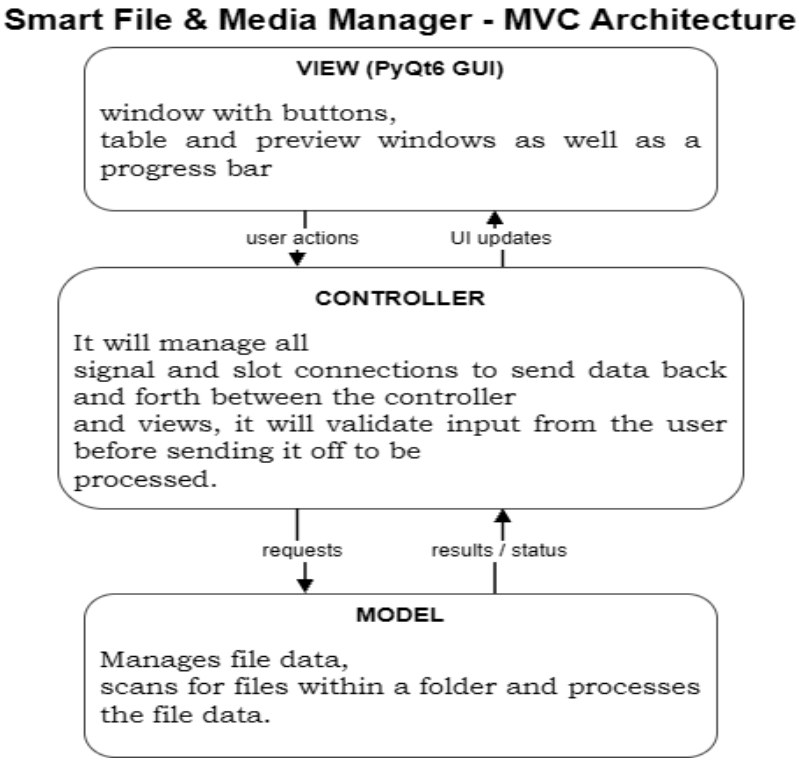
The primary elements of our main interface are all part of a single PyQt6 widget that includes the application's main window as well as buttons to open the folders, begin scanning and exporting files.

The other elements include:

- A QTableWidget where we display the file information
- A QLineEdit for searching files
- A QComboBox to filter files based on their types
- QLabels to show us both text and image previews
- A QProgressBar to indicate how much progress is being made in scanning
- A QStatusBar to give us an idea of what status the app is currently in
- Lastly there is QMediaPlayer to handle all audio playback through QtMultimedia.

1.4 Application Design – MVC

We are going to employ a MVC (Model-View-Controller) architecture for breaking down our app in to a number of large areas.



Model: Manages files information, scanning folders and processes. The main logic of the app is contained here. However, this part does NOT rely on the graphical User Interface.

View: Provides the presentation layer of the application. This includes buttons, tables with files, previewing multimedia content, and showing users that an operation has been completed or is running.

Controller: The Controller portion of the app acts as a bridge between what the user can do within the UI and how these actions effect the Model. The controller uses appropriate Qt signals and slots to inform the View when changes have occurred so it may be updated.

The advantages of this architecture include making it easier to debug, test, and support the application. It also allows us to run time consuming operations independently of GUI operations.